

BP_VehicleAI2 – Descripción limpia del sistema de fitness

1. Función general del Blueprint

El **BP_VehicleAI2** representa cada vehículo inteligente que participa en la simulación. Este blueprint no solo contiene el coche físico, sino también toda la lógica relacionada con su evaluación: calcula el fitness, detecta comportamientos incorrectos, registra estadísticas de depuración y decide cuándo el coche debe morir.

Su objetivo principal es medir si el vehículo está conduciendo correctamente. Para ello, el blueprint suma premios cuando el coche avanza de forma estable, centrada y segura, y resta penalizaciones cuando se comporta de forma peligrosa o incorrecta, por ejemplo al ir demasiado rápido cerca de un obstáculo, moverse hacia atrás, corregir en dirección equivocada, saltarse un semáforo, avanzar cuando debería estar parado o quedarse bloqueado.

El fitness se actualiza constantemente durante la simulación mediante la función `UpdateFitness`. Además, el coche puede morir por diferentes razones, como `LAZY`, `REVERSE`, `REVERSING_WRONG`, `TIMEFINISHED`, colisiones o infracciones de tráfico. Cuando muere, se guarda una línea en `Fitness_Debug.csv` con todos los datos acumulados del vehículo.

Este blueprint trabaja junto con el `BP_AiController`, que proporciona información importante como la velocidad máxima, el throttle, el steering, los sensores de distancia, el estado de semáforos, la existencia de una orden de parada y el punto objetivo de detención.

2. Configuración inicial y variables principales

Configuración de clase

El coche tiene la opción **Reverse as Brake** desactivada. Esto significa que el sistema no interpreta automáticamente la marcha atrás como una acción normal de frenado. Por tanto, si el coche empieza a retroceder, el blueprint puede penalizarlo explícitamente como comportamiento incorrecto.

Variables generales del estado del coche

`Fitness` guarda la puntuación acumulada del vehículo. Empieza en `0.0` y va aumentando o disminuyendo durante la simulación.

`IsDead` indica si el coche ya ha muerto. Cuando está en `true`, el coche deja de actualizar su fitness y no debe volver a ejecutar la lógica de muerte.

`DeathReason` almacena la causa de muerte del coche. Esta información se usa después en los CSV de resumen y depuración.

`AiController` guarda la referencia al controlador del coche. Desde esta referencia se obtienen datos de conducción, sensores, red neuronal, estado de entrenamiento y señales de tráfico.

`GenerationCount` indica la generación a la que pertenece el coche.

`SpawnPoint` guarda el punto de aparición desde el que salió el vehículo.

`TimeAlived` cuenta cuántos segundos lleva vivo el coche. Se incrementa con el temporizador de `CheckStuck`.

`TimeDeath` evita que la muerte del coche se procese más de una vez.

`MaxTimeAlived` define un límite de vida editable, usado especialmente en test.

`LastLocation` almacena la última posición registrada del coche. Se utiliza para detectar si el coche se ha quedado parado.

`TimeSinceLastMove` cuenta cuánto tiempo lleva el coche sin moverse lo suficiente.

`ActualTrafficLight` guarda el semáforo con el que el coche está interactuando.

`MaxSteeringAngle` representa el ángulo máximo de giro usado en la normalización de algunos cálculos.

`ApproachSpeed` representa la velocidad de aproximación usada para escalar recompensas y penalizaciones relacionadas con zonas de parada.

`bIsExecutingUpdateFitness` funciona como protección para evitar que `UpdateFitness` se ejecute de forma reentrante o simultánea mientras ya se está calculando.

Variables de parada, espera y reanudación

`CurrentStopWaitTime` almacena tiempo de espera actual en una zona de parada.

`StopValidationTime` y `YieldValidationTime` guardan tiempos relacionados con la validación de STOP y YIELD.

`WasInStopContextLastCheck` indica si en la comprobación anterior el coche estaba en un contexto de parada, como un STOP, YIELD o semáforo en rojo.

`ResumeGraceActive` indica si está activa una ventana de gracia después de salir de una condición de parada.

`ResumeGraceTime` define cuánto tiempo le queda al coche para reanudar la marcha antes de que se le pueda considerar bloqueado.

`StopCrossingCheckID` y `YieldCrossingCheckID` sirven como identificadores de control para comprobaciones de cruce en STOP y YIELD.

Variables de penalización puntual

`DeltaLazyPenalty` guarda la penalización concreta aplicada cuando el coche no se mueve lo suficiente.

`DeltaCollisionDeathPenalty` guarda la penalización aplicada cuando se produce una colisión mortal.

`ReverseTime` acumula el tiempo que el coche lleva circulando hacia atrás.

Variables de depuración acumulada

El blueprint almacena muchos acumuladores `Debug_...` para poder analizar después qué ha ocurrido durante la vida del coche.

Entre los más importantes están:

- `Debug_Acum_PenaltyVelocidad`, que acumula penalizaciones por velocidad excesiva.
- `Debug_Acum_ReversePenalty`, que acumula penalizaciones por circular marcha atrás.
- `Debug_Acum_LazyPenalty`, que acumula penalizaciones por quedarse parado sin motivo válido.
- `Debug_Acum_PenaltyTrafficViolation`, que acumula penalizaciones por infringir semáforos o señales.
- `Debug_Acum_PenaltyCorrectingWrong`, que acumula penalizaciones por corregir la trayectoria en dirección incorrecta.
- `Debug_Acum_CreepingPenalty`, que acumula penalizaciones por avanzar lentamente cuando debería permanecer parado.
- `Debug_Acum_WaitBonus`, que acumula premios por esperar correctamente.
- `Debug_Acum_NetNormal`, que acumula el fitness positivo normal de conducción.
- `Debug_DeathPenalty`, que guarda la penalización final asociada a la muerte.
- `Debug_Ticks_Normal`, que cuenta cuántas veces se ha aplicado el cálculo normal de fitness.
- `Debug_Ticks_Total`, que cuenta cuántas actualizaciones totales de fitness se han realizado.
- `Debug_Ticks_StopContext`, que cuenta cuántas actualizaciones han ocurrido en contexto de parada.
- `Debug_Ticks_StopContext_TrafficLight`, `Debug_Ticks_StopContext_Stop` y `Debug_Ticks_StopContext_Yield`, que separan los contextos de parada por tipo.
- `Debug_Acum_ThrottlePositive`, que acumula aceleración positiva.
- `Debug_Acum_BrakeInput`, que acumula uso del freno.
- `Debug_Acum_CoastTime`, que mide tiempo sin acelerar ni frenar de forma relevante.
- `Debug_Acum_SteeringInput`, `Debug_Acum_ABS_Steering` y `Debug_Acum_TargetSteering`, que registran el comportamiento del giro.

- `Debug_MutNumChanged`, `Debug_MutAvgMagnitude`, `Debug_MutNumLarge` y `Debug_MutParentFitness`, que almacenan información sobre la mutación del cerebro neuronal recibido por el coche.
- `Debug_NumStopZonesEntered`, que cuenta cuántas zonas de parada ha atravesado el coche.
- `Debug_NotNorm_PeakFitness`, que guarda el mayor fitness bruto alcanzado antes de normalización.

Estas variables no modifican directamente la conducción, pero permiten analizar después el comportamiento del vehículo en los archivos de salida.

3. Inicio del coche: BeginPlay

Cuando el coche aparece en el mundo, primero se ejecuta el `Parent:BeginPlay`.

Después se obtiene su controlador mediante `GetController` y se intenta convertir a `BP_AiController`. Si la conversión es válida, la referencia se guarda en la variable `AiController`.

A continuación se llama a `SetColor`, que asigna el color inicial del coche. Este color puede ayudar a diferenciar visualmente los coches durante la simulación.

Después se obtiene el componente de movimiento del vehículo y se convierte a `ChaosWheeledVehicleMovementComponent`. Esta referencia permite consultar información física del coche, como si está apoyado en el suelo.

Luego se guarda la posición inicial del coche en `LastLocation`. Esta posición será la referencia inicial para detectar si el coche se mueve o se queda bloqueado.

Por último, se crea un temporizador con `SetTimerByEvent`. Este temporizador se ejecuta cada `1.0` segundo y llama al evento `CheckStuck`. El `TimerHandle` se guarda en `CheckStuckTimerHandle`, para poder cancelarlo cuando el coche muera.

4. Actualización por frame: Event Tick

En cada frame, se ejecuta primero el `Parent:Tick`.

Después se ajusta el `AngularDamping` del mesh. Si el vehículo está en el suelo, se aplica un damping mayor; si no está en el suelo, se usa un valor menor. Esto ayuda a controlar la estabilidad física del coche dependiendo de si está apoyado o no.

A continuación se comprueba si el coche sigue vivo. Si `IsDead` es `true`, no se hace nada más.

Si el coche está vivo, el blueprint comprueba si debe finalizar por tiempo:

- Si el coche no está en modo entrenamiento y `TimeAlived` supera `MaxTimeAlived`, se llama a `KillCar` con la causa `TIMEFINISHED`.
- Si no se cumple lo anterior, pero `TimeAlived` supera `120`, también se llama a `KillCar` con la causa `TIMEFINISHED`.
- Si no ha llegado al límite de tiempo, se llama a `UpdateFitness`.

De esta forma, el `Tick` es el encargado de actualizar el fitness continuamente mientras el coche está vivo.

5. Detección de bloqueo: CheckStuck

`CheckStuck` se ejecuta cada segundo mediante el temporizador creado en `BeginPlay`. Su función es detectar si el coche se ha quedado parado sin motivo válido y, en ese caso, penalizarlo o matarlo por `LAZY`.

Primero comprueba si el coche está muerto. Si lo está, se cancela el temporizador usando `Clear and Invalidate Timer by Handle`.

Si el coche sigue vivo, se incrementa `TimeAlived`.

Durante los primeros tres segundos de vida, el sistema no aplica la detección de bloqueo. En ese periodo inicial solo actualiza:

- `WasInStopContextLastCheck`, indicando si el coche está en una situación de parada.
- `TimeSinceLastMove`, poniéndolo a `0`.
- `LastLocation`, guardando la posición actual.

A partir de los tres segundos, el sistema empieza a comprobar cambios de contexto.

Primero detecta si el coche acaba de entrar en una zona de parada. Esto ocurre cuando antes no estaba en contexto de parada y ahora sí lo está. Si sucede, aumenta `Debug_NumStopZonesEntered`.

Después detecta si el coche acaba de salir de una situación de parada. Esto ocurre cuando antes estaba en contexto de parada y ahora ya no lo está.

Cuando el coche sale de una parada, se activa `ResumeGraceActive`. Esta ventana de gracia evita que el coche sea penalizado inmediatamente como bloqueado justo después de que un semáforo se ponga en verde o deje de existir una orden de parada.

El tiempo de gracia se calcula dinámicamente con:

```
ResumeGraceTime = Clamp(SensorEntryDistance / AproachSpeed, 1.0, 3.0)
```

Así, cuanto mayor sea la distancia de entrada o menor sea la velocidad de aproximación, más margen tiene el coche para reanudar la marcha, siempre dentro de un rango limitado entre 1 y 3 segundos.

Ventana de gracia de reanudación

Si `ResumeGraceActive` está activo, el sistema reduce `ResumeGraceTime` en 1.0 cada vez que se ejecuta `CheckStuck`.

Durante este tiempo, también registra qué está haciendo el coche:

- Si `Throttle > 0.0`, acumula aceleración en `Debug_Acum_ThrottleDuringGraceTime`.
- Si `Throttle <= 0.0`, acumula frenado en `Debug_Acum_BrakeDuringGraceTime`.

Cuando `ResumeGraceTime` llega a 0.0 o menos, se desactiva `ResumeGraceActive`.

Después se actualizan `TimeSinceLastMove`, `WasInStopContextLastCheck` y `LastLocation`.

Penalización por quedarse parado

Si no hay ventana de gracia activa, el sistema comprueba si el coche se ha movido menos de lo esperado.

La condición principal compara la distancia entre la posición actual y `LastLocation` con un umbral dinámico basado en la velocidad máxima y la distancia frontal al obstáculo.

El umbral se calcula aproximadamente como:

- `MaxSpeed * 0.05`, si no hay un obstáculo frontal relevante.
- `MaxSpeed * 0.05 * FrontObstacleDistance`, si hay un obstáculo frontal suficientemente lejos.

El coche solo puede ser penalizado por `LAZY` si además no está en `ForceStop` y no está detenido por un semáforo activo en rojo o amarillo.

Si el coche se mueve menos que ese umbral, aumenta `TimeSinceLastMove` y se calcula `DeltaLazyPenalty` como la diferencia entre el movimiento esperado y el movimiento real.

Después se resta esa penalización al fitness:

```
Fitness = Fitness - DeltaLazyPenalty
```

Y se acumula en:

```
Debug_Acum_LazyPenalty
```

Si `TimeSinceLastMove >= 3`, el coche se considera bloqueado. En ese caso, `Debug_DeathPenalty` toma el valor de `DeltaLazyPenalty` y se llama a `KillCar` con la causa `LAZY`.

Si el coche sí se mueve lo suficiente, se reinicia `TimeSinceLastMove`, se actualiza `WasInStopContextLastCheck` y se guarda la nueva `LastLocation`.

6. Colisiones y overlaps: EventActorBeginOverlap

Cuando el coche empieza a solaparse con otro actor, primero se comprueba que `OtherActor` sea válido.

Después se intenta convertir `OtherActor` a `BPIntersectionBorders`. Estos actores representan bordes o límites de intersecciones.

Si la conversión es válida, se comprueba si el borde corresponde al trigger actual del controlador:

```
Trigger del borde == CurrentTrigger del AiController
```

Si esto se cumple, se comprueba también si la intención de dirección del borde coincide con la decisión de intersección del controlador:

```
DirectionIntent del borde == IntersectionDecision del AiController
```

Si ambas condiciones son verdaderas, se considera una colisión o cruce incorrecto dentro de la intersección. Se calcula una penalización de muerte proporcional a la velocidad del coche y a la falta de distancia frontal:

```
DeltaCollisionDeathPenalty = (ForwardSpeed2 / MaxSpeed) * (1 - FrontObstacleDistance)
```

Después se resta esa penalización al fitness, se guarda en `Debug_DeathPenalty` y se llama a `KillCar` con una causa del tipo:

```
COLLISION_WITH_NombreDelActor
```

Si el actor no corresponde a una intersección válida para matar al coche, el sistema comprueba si tiene la etiqueta `Semaforo`. Si la tiene, se convierte a `BP_TrafficLight` y se guarda como `ActualTrafficLight`.

Si la conversión a `BPIntersectionBorders` falla, el sistema comprueba si el actor tiene la etiqueta `Obstaculo` o `Discontinua`. En ese caso, se aplica la lógica de penalización y muerte correspondiente a la colisión. Si no, vuelve a comprobar si el actor es un semáforo.

7. Fin de overlap con semáforo: EventActorEndOverlap

Cuando el coche deja de solaparse con un actor, el sistema intenta convertir ese actor a `BP_TrafficLight`.

Si el semáforo detectado es el mismo que `ActualTrafficLight`, el coche deja de estar asociado a ese semáforo. Por tanto:

1. `ActualTrafficLight` se limpia.
2. `ForceStop` del `AiController` se pone en `false`.

Esto evita que el coche mantenga una orden de parada asociada a un semáforo que ya no está afectándole.

8. Muerte del coche: KillCar

`KillCar` recibe como entrada una cadena llamada `Cause`, que representa la razón de muerte.

Lo primero que hace es incrementar `TimeDeath`. Si `TimeDeath` es mayor que `1`, no continúa. Esta comprobación evita que el mismo coche pueda morir varias veces y escribir varias líneas duplicadas en el CSV.

Si es la primera muerte real del coche, el sistema:

1. Resta `1` a `TimeAlived`.
2. Pone `IsDead` en `true`.
3. Pone `bIsExecutingUpdateFitness` en `false`.
4. Guarda `Cause` en `DeathReason`.
5. Desactiva la simulación física del mesh.
6. Cambia el color del coche a rojo.

Después escribe una línea en `Fitness_Debug.csv`. Esta línea contiene toda la información relevante del coche:

- Generación.
- Nombre del coche.
- Fitness final.
- Tiempo vivo.
- Spawn point usado.
- Razón de muerte.
- Penalización de muerte.
- Penalizaciones acumuladas.
- Componentes del fitness normal.
- Premios de espera.
- Penalizaciones por tráfico, reversa, creeping y lazy.
- Estadísticas de throttle, freno y steering.

- Tiempos de validación de STOP y YIELD.
- Información sobre mutaciones.
- Número de zonas de parada recorridas.
- Pico de fitness bruto alcanzado.

El archivo se guarda con `AllowOverwriting = false`, por lo que cada coche añade una nueva línea al CSV en lugar de sobrescribirlo.

9. Actualización del fitness: UpdateFitness

`UpdateFitness` es la función principal de evaluación del coche. Se ejecuta mientras el vehículo está vivo y calcula tanto premios como penalizaciones.

Al inicio, la función comprueba que no se esté ejecutando ya y que el coche no esté muerto:

```
NOT(bIsExecutingUpdateFitness OR IsDead)
```

Si la condición es válida, pone `bIsExecutingUpdateFitness` en `true` para bloquear ejecuciones simultáneas.

Después calcula `SafeDelta`, que es el tiempo entre frames limitado entre `0.001` y `0.1`:

```
SafeDelta = Clamp(GetWorldDeltaSeconds, 0.001, 0.1)
```

Este valor se usa para que las recompensas y penalizaciones sean proporcionales al tiempo y no dependan directamente de la tasa de frames.

10. Penalización por velocidad excesiva

La primera penalización comprueba si el coche va demasiado rápido teniendo en cuenta la distancia frontal al obstáculo.

La condición es:

```
ForwardSpeed >= MaxSpeed * FrontObstacleDistance
```

`FrontObstacleDistance` funciona como un valor normalizado donde `1` significa que el obstáculo está lejos y `0` significa que está cerca. Por tanto, cuanto más cerca está el obstáculo, menor debería ser la velocidad permitida.

Si el coche supera esa velocidad adaptada, se calcula:

```
DeltaPenaltyVelocidad = max(0, Clamp(ForwardSpeed / MaxSpeed, 0, 1) - FrontObstacleDistance)2 * SafeDelta * MaxSpeed
```

Esta penalización aumenta cuando el coche mantiene una velocidad alta cerca de un obstáculo.

Después se resta al fitness y se acumula en `Debug_Acum_PenaltyVelocidad`.

11. Penalización por marcha atrás

Después se comprueba si el coche circula hacia atrás:

```
ForwardSpeed < 0.0
```

Si no va marcha atrás, `ReverseTime` se reinicia a `0.0`.

Si sí va marcha atrás, se incrementa `ReverseTime` con `SafeDelta` y se calcula una penalización proporcional a la velocidad negativa:

```
DeltaReversePenalty = Clamp(abs(ForwardSpeed) / MaxSpeed, 0, 1)2 * SafeDelta * MaxSpeed
```

Esta penalización se resta del fitness y se acumula en `Debug_Acum_ReversePenalty`.

12. Detección de corrección incorrecta y marcha atrás grave

Después de penalizar la marcha atrás, el sistema comprueba si el coche está realizando alguno de estos comportamientos peligrosos:

1. Está corrigiendo la dirección hacia el lado equivocado cuando no hay obstáculo frontal cercano.
2. Está yendo marcha atrás durante demasiado tiempo o demasiada distancia.

La corrección incorrecta se detecta con esta idea:

```
Steering * HorizontalDistDiff < 0.0
```

`HorizontalDistDiff` es la diferencia entre la distancia del sensor derecho y la distancia del sensor izquierdo. Si el producto con el steering es negativo, significa que el coche está girando en una dirección que no ayuda a centrarse.

Además se exige que:

- `FrontObstacleDistance >= 0.99`
- `abs(HorizontalDistDiff) > 0.075`

Esto evita penalizar correcciones pequeñas o situaciones donde hay un obstáculo frontal relevante.

La marcha atrás grave se detecta cuando:

- `ForwardSpeed < -5.0`
- La distancia recorrida marcha atrás en relación con la distancia visible supera un umbral dinámico basado en la velocidad actual y la velocidad de aproximación.

Si se cumplen ambos problemas a la vez, el coche recibe una penalización de muerte:

```
DeltaDeathPenalty = abs(ForwardSpeed) + MaxVisionDistance * abs(Steering) *  
abs(HorizontalDistDiff)
```

Después muere con la causa `REVERSING_WRONG`.

Si solo se cumple la condición grave de marcha atrás, la penalización es:

```
DeltaDeathPenalty = abs(ForwardSpeed)
```

Y el coche muere con la causa `REVERSE`.

Si el problema detectado es solamente una corrección incorrecta, no se mata al coche inmediatamente. En ese caso se calcula:

```
DeltaPenaltyCorrectingWrong = MaxVisionDistance * abs(Steering) *  
abs(HorizontalDistDiff) * SafeDelta
```

Esta penalización se resta del fitness y se acumula en `Debug_Acum_PenaltyCorrectingWrong`.

13. Contexto de parada: semáforos, STOP y YIELD

Si el coche no entra en una condición de muerte por marcha atrás o corrección grave, pasa al cálculo normal del fitness.

Primero se comprueba si está en un contexto donde debe parar:

```
ForceStop OR (IsTrafficLightActive AND TrafficLightState <= 0.5)
```

Esta condición agrupa situaciones como STOP, YIELD o semáforo no favorable.

Penalización por infracción de semáforo

Si el coche está ante un semáforo activo y el estado del semáforo es menor o igual a `0.5`, se aplica una penalización por seguir avanzando cuando debería detenerse.

La fórmula es:

```
DeltaPenaltyTrafficViolation = Clamp(ForwardSpeed / MaxSpeed, 0, 1)2 * SafeDelta * MaxSpeed * FactorSemaforo
```

El `FactorSemaforo` depende del estado del semáforo:

- Si el estado es `0.5`, se usa un factor menor.
- En otros estados de parada, se usa un factor mayor.

Después se resta al fitness y se acumula en `Debug_Acum_PenaltyTrafficViolation`.

Penalización por girar mientras debería estar parado

Dentro del contexto de parada, se comprueba si el coche está girando en una dirección incorrecta según la diferencia lateral de sensores:

```
Steering * HorizontalDistDiff < 0.0
```

Si se cumple, se calcula:

```
DeltaSteeringWhileStopped = abs(Steering) * SafeDelta * MaxSpeed
```

Esta penalización castiga que el coche siga corrigiendo o girando de forma inadecuada cuando debería estar detenido.

Se resta del fitness y se acumula en `Debug_Acum_SteeringWhileStopped`.

Penalización por aproximarse girando demasiado

Después se calcula `DeltaSteerApproachPenalty`, una penalización que castiga aproximarse a una zona de parada con velocidad, steering y velocidad de aproximación elevadas.

La fórmula combina:

- Velocidad actual normalizada.
- Velocidad máxima.
- Velocidad de aproximación normalizada.
- Valor absoluto del steering.

- `SafeDelta`.

Conceptualmente, cuanto más rápido se acerca el coche y más gira en ese momento, mayor será la penalización.

Después se resta del fitness y se acumula en `Debug_Acum_SteerApproachPenalty`.

Penalización por creeping

Si el tipo de control de tráfico es STOP o semáforo, y además existe una orden de parada activa, se calcula una penalización por `creeping`.

El `creeping` representa avanzar lentamente cuando el coche debería permanecer detenido.

La penalización combina:

- La velocidad actual en valor absoluto.
- La velocidad de aproximación.
- `SafeDelta`.
- La cercanía al `StopTargetLocation`.
- La distancia de entrada del sensor.

La parte de cercanía al objetivo hace que la penalización sea mayor cuando el coche está más cerca del punto donde debería haberse detenido.

Después se resta del fitness y se acumula en `Debug_Acum_CreepingPenalty`.

Premio por esperar correctamente

Si el coche está en contexto de parada y su velocidad es prácticamente cero, se concede un premio de espera.

La condición de velocidad parada es:

```
abs(ForwardSpeed) < MaxSpeed * 0.005
```

Cuando se cumple, se calcula `DeltaWaitBonus`. Este premio valora que el coche esté detenido correctamente y combina:

- La alineación lateral, usando la diferencia entre los sensores derecho e izquierdo.
- La distancia frontal al obstáculo.
- `SafeDelta`.
- La velocidad de aproximación.
- La relación entre `ApproachSpeed` y `MaxSpeed`.

- La cercanía al `StopTargetLocation`.

El premio aumenta si el coche está parado, bien alineado y cerca del punto correcto de detención.

Después se suma al fitness y se acumula en `Debug_Acum_WaitBonus`.

14. Fitness normal de conducción

Después de las penalizaciones y premios de parada, se calcula el fitness normal de conducción. Este fitness representa el comportamiento deseado cuando el coche avanza por la carretera.

El cálculo se divide en varios componentes: `DeltaA`, `DeltaB`, `DeltaC`, `DeltaD`, `DeltaE`, `DeltaF` y `DeltaNetNormal`.

DeltaA: avance seguro y alineado

`DeltaA` premia avanzar hacia delante, pero teniendo en cuenta la alineación lateral y la distancia frontal.

La fórmula es:

```
DeltaA = ForwardSpeed * LateralAlignment * FrontObstacleDistance * SafeDelta
```

El factor `LateralAlignment` se calcula mediante un `LERP` entre:

- `1.0`
- `1 - abs(DistSensorRight - DistSensorLeft)`

Esto significa que el avance vale más si el coche está centrado entre los laterales de la carretera.

DeltaB: mejora del centrado lateral

`DeltaB` mide si el coche ha mejorado su centrado respecto al tick anterior.

Se calcula como:

```
DeltaB = max(0, PrevHorizontalDistanceDifference - abs(HorizontalDistDiff))
```

Si la diferencia lateral actual es menor que la anterior, el coche se ha centrado mejor y obtiene un valor positivo. Si ha empeorado o no ha mejorado, el valor es `0`.

DeltaC: normalización por anchura de carretera

DeltaC normaliza el efecto de DeltaB según la anchura de la carretera:

$$\text{DeltaC} = 1 / \max(\text{RoadWidth}, 0.05)$$

Así, la mejora lateral se interpreta de forma proporcional al tamaño de la carretera.

DeltaD: normalización por capacidad máxima de giro

DeltaD utiliza el ángulo máximo de steering para normalizar el efecto del giro:

$$\text{DeltaD} = 1 / \sin(\text{MaxSteeringAngle en radianes})$$

Esto permite escalar el cálculo según la capacidad real de giro del vehículo.

DeltaE: mejora lateral normalizada

DeltaE combina los tres valores anteriores:

$$\text{DeltaE} = \text{DeltaB} * \text{DeltaC} * \text{DeltaD}$$

Por tanto, representa cuánto ha mejorado el centrado lateral, pero normalizado por la anchura de la vía y por la capacidad máxima de giro.

DeltaF: velocidad segura respecto al obstáculo frontal

DeltaF premia que la velocidad del coche esté adaptada a la distancia frontal.

La fórmula compara la velocidad normalizada con FrontObstacleDistance. Si ambas están equilibradas, el coche recibe más fitness. Si va demasiado rápido para la distancia disponible, el valor disminuye.

También multiplica por:

- Velocidad máxima.
- $\text{FrontObstacleDistance}^2$.
- SafeDelta.
- El máximo entre velocidad normalizada y throttle positivo.

Conceptualmente, DeltaF premia avanzar con una velocidad útil, pero no excesiva para la distancia libre disponible.

DeltaNetNormal: fitness positivo final

El fitness normal final se calcula como:

```
DeltaNetNormal = DeltaA * DeltaE * DeltaF
```

Después se suma al fitness total:

```
Fitness = Fitness + DeltaNetNormal
```

Este valor representa el premio principal por conducir bien: avanzar, mantenerse centrado, mejorar la trayectoria y adaptar la velocidad a la situación frontal.

Después se acumulan todos los componentes en sus variables de debug:

- Debug_Acum_A
- Debug_Acum_B_Sum
- Debug_Acum_C
- Debug_Acum_D
- Debug_Acum_E
- Debug_Acum_F
- Debug_Acum_NetNormal

También se incrementa Debug_Ticks_Normal .

15. Registro de estadísticas de entrada y conducción

Al final de UpdateFitness , se incrementa Debug_Ticks_Total y se acumulan varias métricas de conducción.

Primero se guardan estadísticas de steering:

- Debug_Acum_SteeringInput suma el steering real aplicado.
- Debug_Acum_ABS_Steering suma el valor absoluto del steering.
- Debug_Acum_TargetSteering suma el steering objetivo calculado por el controlador.

Después se registran throttle y freno:

- Si Throttle > 0.0 , se acumula en Debug_Acum_ThrottlePositive .
- Si Throttle <= 0.0 , se acumula en Debug_Acum_BrakeInput usando el valor absoluto.

También se mide el tiempo en coast. Si $\text{abs}(\text{Throttle}) \leq 0.05$, se suma SafeDelta a Debug_Acum_CoastTime .

16. Registro de contexto de parada

Si el coche está en contexto de parada, se actualizan contadores específicos.

La condición de contexto de parada es:

```
ForceStop OR (IsTrafficLightActive AND TrafficLightState <= 0.5)
```

Cuando se cumple, el sistema mira `TrafficControlType`:

- Si `TrafficControlType == 0`, incrementa `Debug_Ticks_StopContext_TrafficLight`.
- Si `TrafficControlType == 1`, incrementa `Debug_Ticks_StopContext_Stop`.
- En cualquier otro caso, incrementa `Debug_Ticks_StopContext_Yield`.

Después incrementa `Debug_Ticks_StopContext`.

También registra qué hace el coche en ese contexto:

- Si `Throttle > 0.0`, acumula aceleración en `Debug_Acum_StopContextThrottle`.
- Si `Throttle <= 0.0`, acumula frenado en `Debug_Acum_StopContextBrake`.

Esto permite analizar si el coche acelera o frena cuando debería estar detenido.

17. Cierre de UpdateFitness

Al final de la función, se actualiza `PrevHorizontalDistanceDifference` con el valor absoluto de la diferencia entre el sensor derecho y el sensor izquierdo:

```
PrevHorizontalDistanceDifference = abs(DistSensorRight - DistSensorLeft)
```

Después se actualiza `Debug_NotNorm_PeakFitness` con el máximo entre su valor anterior y el fitness actual:

```
Debug_NotNorm_PeakFitness = max(Debug_NotNorm_PeakFitness, Fitness)
```

Por último, se pone `bIsExecutingUpdateFitness` en `false`, permitiendo que la función pueda ejecutarse de nuevo en el siguiente tick.

18. Resumen del sistema de fitness

El sistema de fitness del **BP_VehicleAI2** está diseñado para evaluar el comportamiento completo del coche durante su vida.

Por un lado, premia la conducción correcta. El coche obtiene fitness positivo cuando avanza, se mantiene centrado, mejora su posición lateral, adapta su velocidad a la distancia frontal y espera correctamente en zonas donde debe detenerse.

Por otro lado, penaliza comportamientos peligrosos o no deseados. El coche pierde fitness si va demasiado rápido cerca de obstáculos, circula marcha atrás, corrige hacia el lado incorrecto, gira cuando debería estar parado, avanza lentamente en una zona de parada, infringe señales de tráfico o se queda bloqueado sin motivo válido.

Además, algunas situaciones provocan muerte inmediata, como quedarse parado demasiado tiempo, circular marcha atrás de forma grave, corregir mal mientras retrocede, chocar con un borde de intersección o superar el tiempo máximo de vida.

Cada vez que un coche muere, el blueprint guarda toda la información acumulada en `Fitness_Debug.csv`. Esto permite analizar después no solo el fitness final, sino también qué partes del cálculo han contribuido más, qué penalizaciones han sido más frecuentes y qué tipo de error ha provocado la muerte del vehículo.

En conjunto, este blueprint convierte la conducción del vehículo en una puntuación numérica útil para el sistema evolutivo. Esa puntuación permite al `BP_EvoLutionManager` comparar coches, seleccionar los mejores modelos neuronales y continuar el proceso de entrenamiento o evaluación.